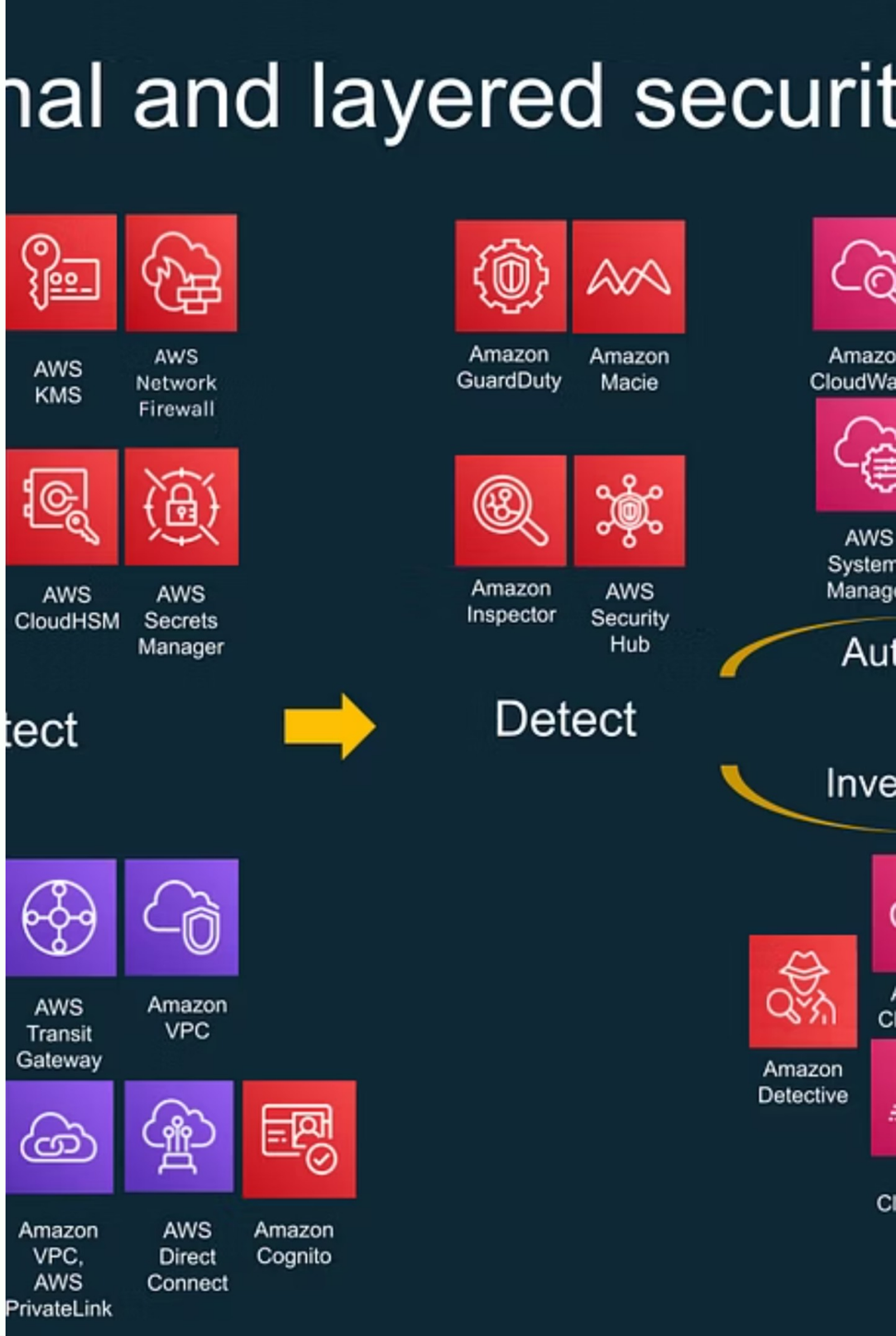
AWS Identity and Access Management (IAM)

Deep Dive: Complete Guide to IAM Fundamentals, Federation, and CI/CD Security



What is AWS IAM?

Core Security Service

Control who can do what with which AWS resources across all services and regions

Global Service

Same IAM applies across all AWS regions, available in every account by default

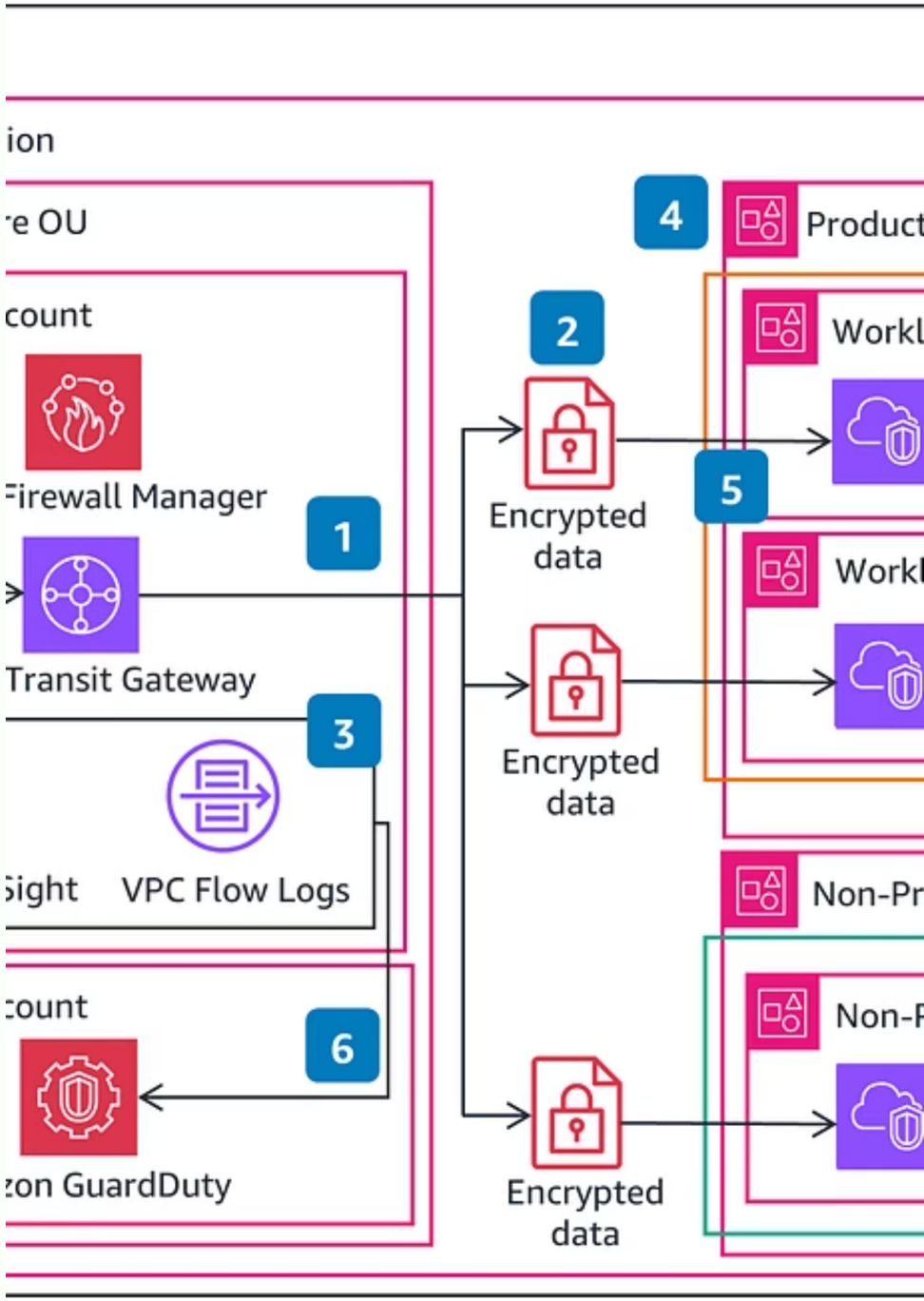
No Cost

IAM is free; you only pay for resources accessed through IAM permissions

Federation Support

Supports integration with external identity providers for centralized authentication

IAM is the foundation of AWS security. Every resource access is controlled through IAM policies, making it essential for anyone working with AWS.



IAM Core Components

IAM Users

Individual identity with unique credentials. Can have programmatic or console access. Long-lived credentials requiring rotation.

IAM Groups

Container for IAM users to simplify permission management. Not used for service-to-service authentication.

IAM Roles

Temporary credentials with STS tokens. Assumed by services or users. Session-based, preferred for modern architectures.

Policies

JSON documents defining permissions. Attached to users, groups, or roles. Evaluated during API calls.

Component	Permanence	Use Case	Credentials
IAM User	Permanent	Apps outside AWS	Long-lived keys
Role	Temporary	EC2, Lambda, cross-account	STS tokens
Group	Permanent	Organizing users	None
Policy	Permanent	Define permissions	None

IAM Policy Structure

Policy JSON Anatomy

Version: Always "2012-10-17" (policy language version)

Statement: Array of permission statements, evaluated independently

Effect: Either "Allow" or "Deny" - explicit deny always wins

Action: AWS API actions (e.g., "s3:GetObject", "ec2:*")

Resource: ARNs affected by the action

Condition: Optional restrictions (IP, time, MFA, region)

Policy Evaluation Logic

- Default: All actions denied
- Explicit allow: Permits action
- Explicit deny: Blocks action (cannot be overridden)
- No matching policy: Request denied

Small policy changes can have huge security implications. Understanding this structure is essential.

```
{ "Version": "2012-10-17", "Statement": [{ "Sid": "AllowS3ReadAccess", "Effect": "Allow", "Action": ["s3:GetObject", "s3:ListBucket"], "Resource": ["arn:aws:s3:::my-bucket/*"], "Condition": { "IpAddress": {"aws:SourceIp": "10.0.0.0/8"} } }]}
```

Types of IAM Policies

01

Identity-Based Policies

Attached to users, groups, or roles. Define what that identity can do. Most common policy type.

02

Resource-Based Policies

Attached to resources like S3, SNS, Lambda. Define who can access the resource. Inline only.

03

Permission Boundaries

Maximum permission ceiling. Prevents privilege escalation. Does not grant access by itself.

04

Session Policies

Temporary policies during credential generation. Cannot expand permissions beyond identity policy.

05

Service Control Policies

Organization-level controls. Filtering only, cannot grant permissions. Affects all principals in account.

Access allowed if: explicit allow in identity policy AND no explicit deny in any policy. Explicit deny always wins.

Permissions Boundaries Explained

What It Is

A maximum permission ceiling that filters other policies. Cannot grant access by itself - only restricts.

How It Works

Effective Permissions = Identity Policy AND Permissions Boundary. Intersection model ensures safety.

Key Use Case

Delegated administration. Developers can create roles, but boundary prevents over-privileging.

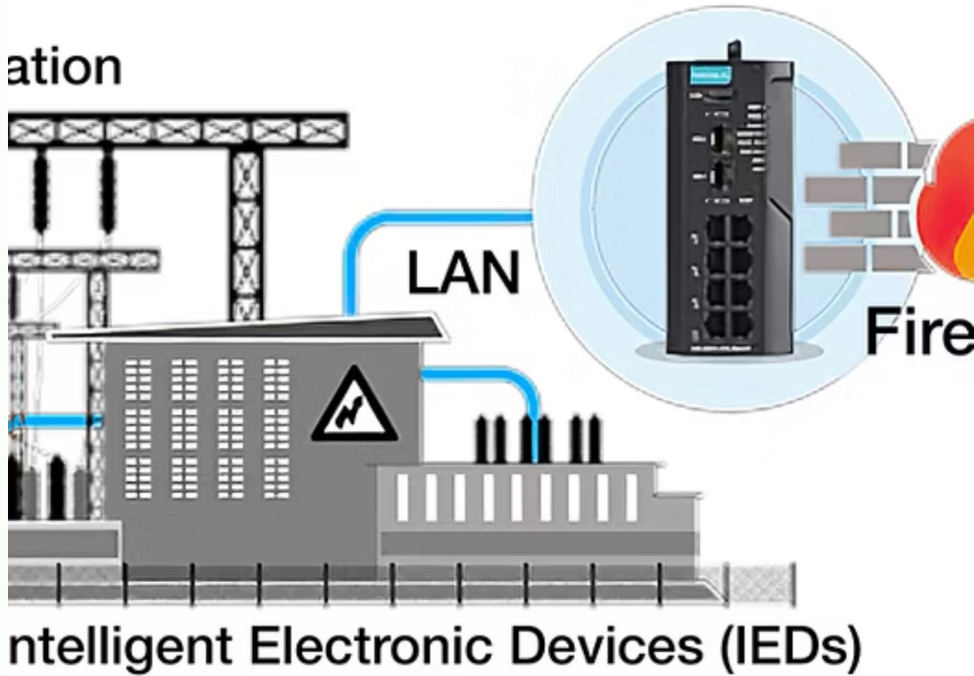
Example Scenario

Developer has administrator policy attached, but permissions boundary limits to EC2 in us-east-1 and S3 read only.

Result: Developer cannot access RDS, networking, IAM, or other regions despite admin policy.

Benefits

- Prevents accidental over-privileging
- Allows self-service role creation
- Maintains central security guardrails
- Simplifies compliance



Implementing Permissions Boundaries

Design the Boundary

Decide maximum permissions: which services, resources, and regions are allowed.

Create the Policy

Create AWS managed or customer managed policy defining the ceiling.

Create Role with Boundary

During role creation, select permissions boundary and attach identity policies.

Monitor Effectiveness

Use IAM Access Analyzer, CloudTrail logs, and policy simulator to verify.

Developer Self-Service Pattern

Central admin creates boundary policy. Developers get CreateRole permission but can only create roles WITH the boundary. Developers attach policies, but limited by boundary. Central admin can remove overprovisioned permissions later.

```
{ "Effect": "Allow", "Action": ["iam:CreateRole", "iam:AttachRolePolicy"], "Resource": "arn:aws:iam::ACCOUNT-ID:role/*", "Condition": { "StringEquals": { "iam:PermissionsBoundary": "arn:aws:iam::ACCOUNT-ID:policy/DeveloperBoundary" } }}
```

Identity vs Resource-Based Policies

Aspect	Identity-Based	Resource-Based
Attached To	Users, Groups, Roles	Resources (S3, SNS, Lambda)
Specifies	What principal can do	Who can access resource
Question	What can I do?	Who can access me?
Use Case	Daily access control	Cross-account sharing
Type	Managed or inline	Inline only
Cross-account	Identity + Resource both needed	Resource specifies cross-account principal

When to Use Identity-Based

- Employee access within same account
- Standard AWS operations
- Most common scenario
- Simpler to understand

When to Use Resource-Based

- Sharing resources across accounts
- Third-party access
- Service-to-service access
- Clear visibility of resource access

The Principle of Least Privilege

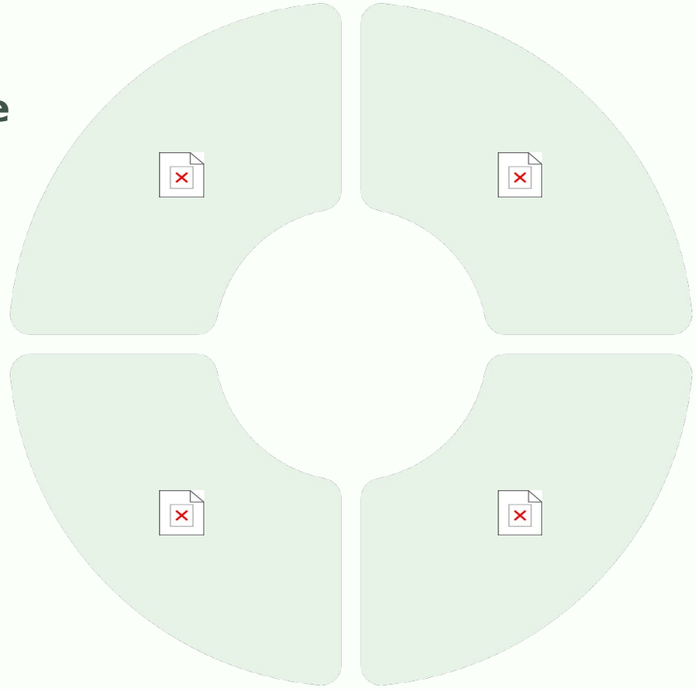
Grant Only Minimum Permissions Necessary

Reduces Attack Surface

Fewer permissions mean fewer ways an attacker can misuse compromised credentials.

Limited Blast Radius

Breach of least-privilege account limits attacker to specific resources only.



Compliance Requirements

HIPAA, PCI-DSS, SOC 2 all require least privilege implementation.

Operational Benefits

Easier troubleshooting, clearer intent, simpler onboarding and offboarding.

Implementing Least Privilege

Start Broad

Begin with broader policy, run workload, monitor CloudTrail for actual actions used.

Analyze Usage

Use Access Analyzer to identify unused permissions and actual usage patterns.

Refine Policy

Create policy based on actual usage, remove extra permissions iteratively.

Continuous Review

Regular audits, quarterly reviews, update as workload changes.

Common Mistakes to Avoid

Too Broad

```
{"Effect": "Allow", "Action": "*", "Resource": "*"}
```

✓ Specific

```
{"Effect": "Allow", "Action": ["ec2:RunInstances"], "Resource":
"arn:aws:ec2:us-east-1:*:instance/*"}
```

Tools: AWS Access Analyzer identifies unused permissions. CloudTrail logs all API actions. IAM Policy Simulator tests policy combinations.

Introduction to Federation

What is Federation?

Allowing users to authenticate via external identity provider, receiving temporary AWS credentials.

No IAM Users Needed

No need for AWS IAM users or long-lived credentials. Central authentication through IdP.

Automatic Expiration

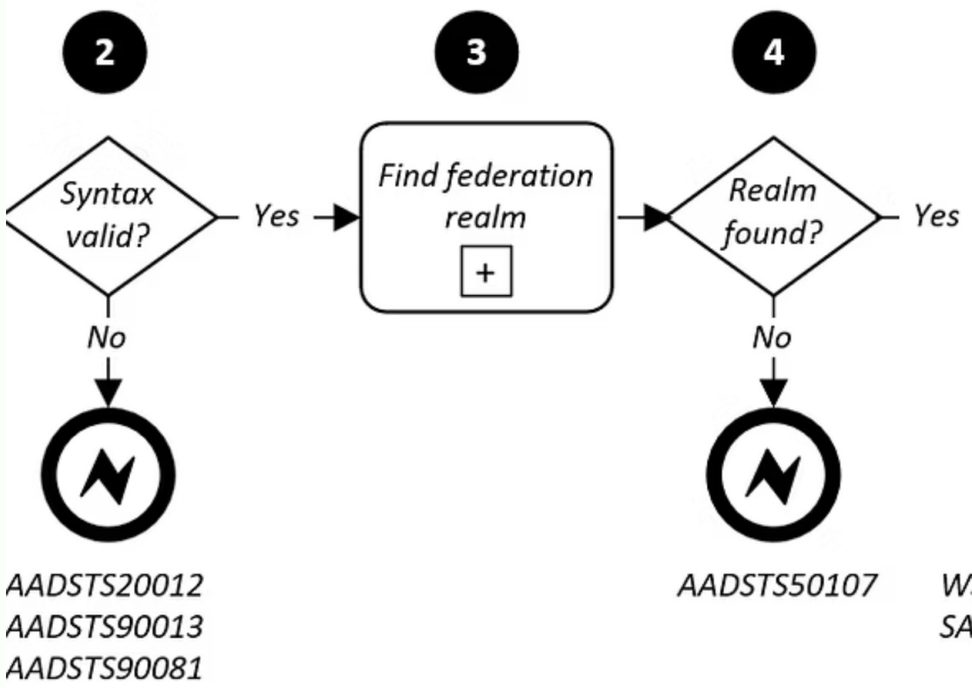
Temporary credentials expire automatically (default 1 hour). No manual rotation required.

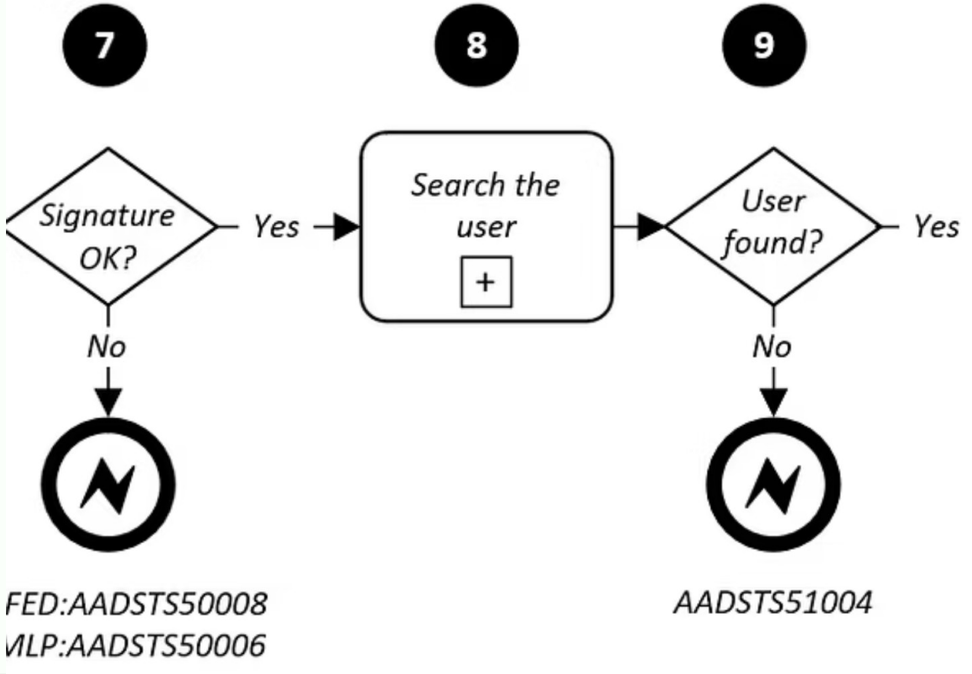
Before Federation

1. Create IAM user in every account
2. Issue access keys or password
3. User authenticates directly to AWS
4. Long-lived credentials stored externally
5. Manual credential rotation

With Federation

1. User exists in external IdP
2. User authenticates to IdP
3. IdP asserts identity to AWS
4. AWS grants temporary credentials
5. Automatic token expiration





and Access Managem

- Authentication
- Authorization
- User Federation
- Social Logins
- Password Management
- Token Management
- Privileged Access
- Identity Brokering

- User Management
- Role Based Provisioning
- Access Governance
- Role Management
- Identity Intellig

- Identity Store
- Internal User Directory
- External User / Customer Direcorty
- Directory Federation

Federation Benefits

Security

No long-lived AWS credentials outside AWS. Centralized authentication and password policy. Easier credential rotation. Single sign-on capability.

Compliance

Meets regulatory requirements (HIPAA, PCI-DSS, SOC 2). Audit trail of authentication. Consistent security policies across organization.

Operational

Simplified onboarding and offboarding. Employee directory as source of truth. Reduces credential management burden significantly.

User Experience

Single sign-on experience. Works with existing corporate credentials. No additional passwords to remember.

Federation Architecture Flow

User → Authenticates to IdP → IdP issues token → AWS STS (token exchange) → AWS issues temporary credentials → User accesses AWS resources

AWS Identity Center (SSO)

1

Multi-Account Access

Grant access to users across multiple AWS accounts with single console entry point and user portal.

2

Identity Source Options

Built-in directory or external connections to Azure AD, Okta, Google Workspace with automatic sync.

3

Permission Sets

Define roles and permissions applied across multiple accounts. Automatically creates IAM roles.

4

ABAC Support

Tag-based access control using user attributes from IdP for dynamic permission assignment.

Aspect	Identity Center	SAML Federation
Setup	Simple (managed)	Complex (manual)
Multi-account	Built-in	Requires role per account
User Portal	Yes (AWS Access Portal)	No (depends on IdP)
Onboarding	Minutes	Hours

AWS Identity Center is AWS's recommended approach for most organizations, combining simplicity with enterprise flexibility.

SAML 2.0 Federation

Security Assertion Markup Language

SAML 2.0 is the enterprise standard for authentication. XML-based protocol used across enterprise software and SaaS applications.

1. **User Requests Access**
   User clicks sign-in link to AWS Management Console
2. **Redirect to IdP**
   AWS sends SAML AuthnRequest, browser redirects to IdP login page
3. **User Authenticates**
   User provides username, password, and MFA to IdP
4. **SAML Assertion Created**
   IdP creates signed SAML assertion (XML document)
5. **Assertion Sent to AWS**
   Browser POSTs assertion to AWS, AWS validates signature
6. **Temporary Credentials Issued**
   AWS issues temporary credentials via AssumeRoleWithSAML
7. **User Accesses Resources**
   User accesses AWS using temporary credentials

Setting Up SAML in AWS

01

Create SAML Identity Provider

Go to IAM → Identity Providers. Upload IdP metadata XML and verify thumbprint.

02

Create Trust Policy

Define which IdP can assume which roles using AssumeRoleWithSAML action.

03

Create IAM Role

Create role with trust policy and attach identity policies defining permissions.

04

Configure IdP

In Okta or Azure AD, create AWS app integration and map users to roles.

05

Test SAML Flow

Use AWS test URL, verify role assumption, check CloudTrail logs.

```
{  "Version": "2012-10-17",  "Statement": [{    "Effect": "Allow",    "Principal": {      "Federated": "arn:aws:iam::123456789012:saml-provider/CompanyIdP"
 },    "Action": "sts:AssumeRoleWithSAML",    "Condition": {      "StringEquals": {"SAML:aud": "https://signin.aws.amazon.com/saml"}    }  }]}
```

SAML is complex but mature and widely supported. Best for enterprises with existing IdP infrastructure.

OpenID Connect (OIDC) Federation

Modern Machine-to-Machine Authentication

What is OIDC?

Modern identity protocol built on OAuth 2.0. JSON-based (simpler than SAML XML). Designed for cloud-native apps.

JWT Tokens

JSON Web Tokens contain user identity and claims. Digitally signed by IdP. Expire after short time.

No Stored Credentials

Completely eliminates long-lived credentials from CI/CD systems. Temporary tokens only.

Aspect	OIDC	SAML
Format	JSON (JWT tokens)	XML (assertions)
Complexity	Simpler	More complex
Use Case	Cloud-native, CI/CD	Enterprise, traditional
Setup Time	Minutes	Hours
IdP Support	GitHub, GitLab, Google	Okta, Azure AD

GitHub Actions + AWS OIDC

Secure CI/CD Without Stored Credentials

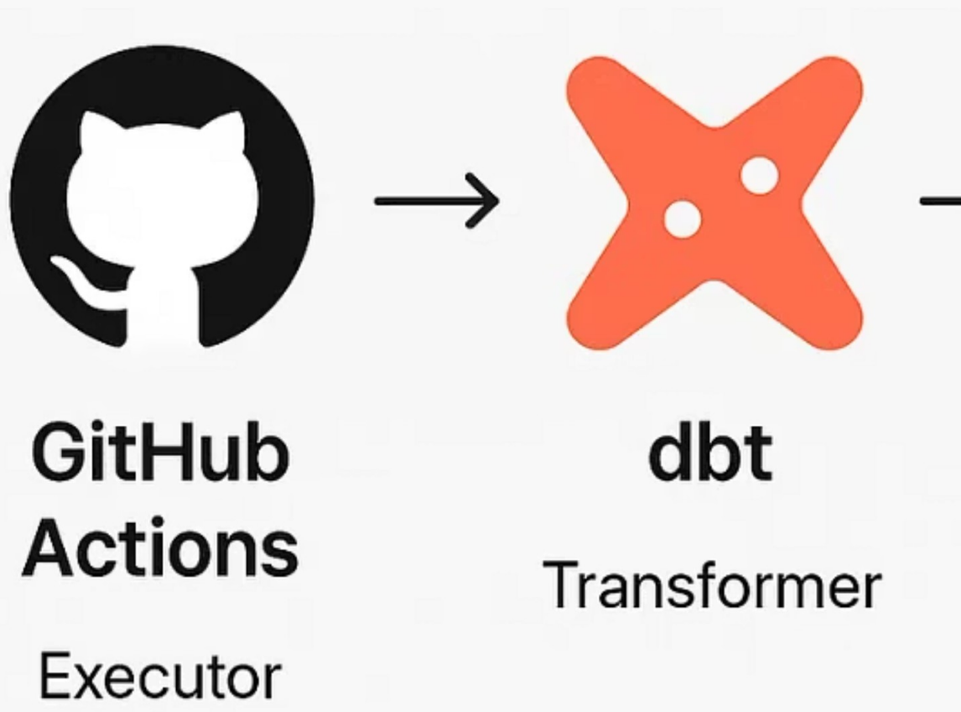
Before OIDC (Bad Practice)

1. Generate AWS access keys
2. Store in GitHub secrets
3. Keys valid for years
4. If leaked: Full account access
5. Manual rotation every 90 days

With OIDC (Best Practice)

1. No AWS credentials stored
2. GitHub issues temporary JWT
3. Exchange JWT for credentials
4. Credentials expire in 1 hour
5. No manual rotation needed

```
name: Deploy to AWSon: pushjobs:  deploy:    permissions:      id-token: write    steps:
   - uses: aws-actions/configure-aws-credentials@v2        with:          role-to-assume:
arn:aws:iam::123456789012:role/GitHubActionsRole          aws-region: us-east-1      -
run: aws s3 cp app.jar s3://my-bucket/
```



OIDC completely eliminates long-lived credentials from CI/CD systems. From a security perspective, this is a huge win.

Comparing Federation Methods

Feature	Identity Center	SAML 2.0	OIDC	API Keys
Setup	Simple	Complex	Simple-Medium	Trivial
Time to Deploy	Hours	Days	Hours	Minutes
Credentials	Managed temp	Signed assertion	JWT token	Long-lived
Expiration	Automatic	Automatic	Automatic	Manual
Multi-account	Native	Requires setup	Per-role	Per-account
Best For	Enterprise multi-account	Enterprise IdP	CI/CD, modern apps	Testing only
Recommended?	✓✓✓ Yes	✓✓ Yes	✓✓✓ Yes (CI/CD)	✗ No

- **Choose Identity Center for multi-account AWS environments with centralized management needs**
- **Choose SAML 2.0 if you already have enterprise IdP like Okta or Azure AD**
- **Choose OIDC for GitHub Actions, GitLab CI, and cloud-native applications**
- **Avoid API Keys - use only for temporary testing with secure storage**

Cross-Account Access Fundamentals

Connecting Resources Across AWS Accounts

Cross-account access enables users or services in one AWS account to access resources in another account securely using IAM roles and trust policies.

Trusted Account

Where the principal (user/role) exists. The "source" requesting access. Example: Account B contains the user.

Trusting Account

Where the resources exist. The "target" for access. Example: Account A contains resources.

Trust Relationship

IAM role in trusting account allows assumption by trusted account. Two-policy requirement ensures security.

Typical Scenarios

Multi-Account Organization: Management, Development, Staging, Production, Logging accounts

Delegated Administration: Central IT team manages multiple business unit accounts

Third-Party Access: Partner company manages part of your infrastructure

Cross-Account Access Methods

Role Assumption

User assumes role in other account. Gets temporary credentials. Most secure and auditable. Recommended method.

Resource-Based Policy

Attach policy to resource (S3, SNS). Grant cross-account principal access. No role assumption. Simpler but less controlled.

External ID

Role trust policy includes external ID. Prevents confused deputy problem. Recommended for third-party access.

Policy Evaluation for Cross-Account

For cross-account access to succeed, BOTH conditions must be true:

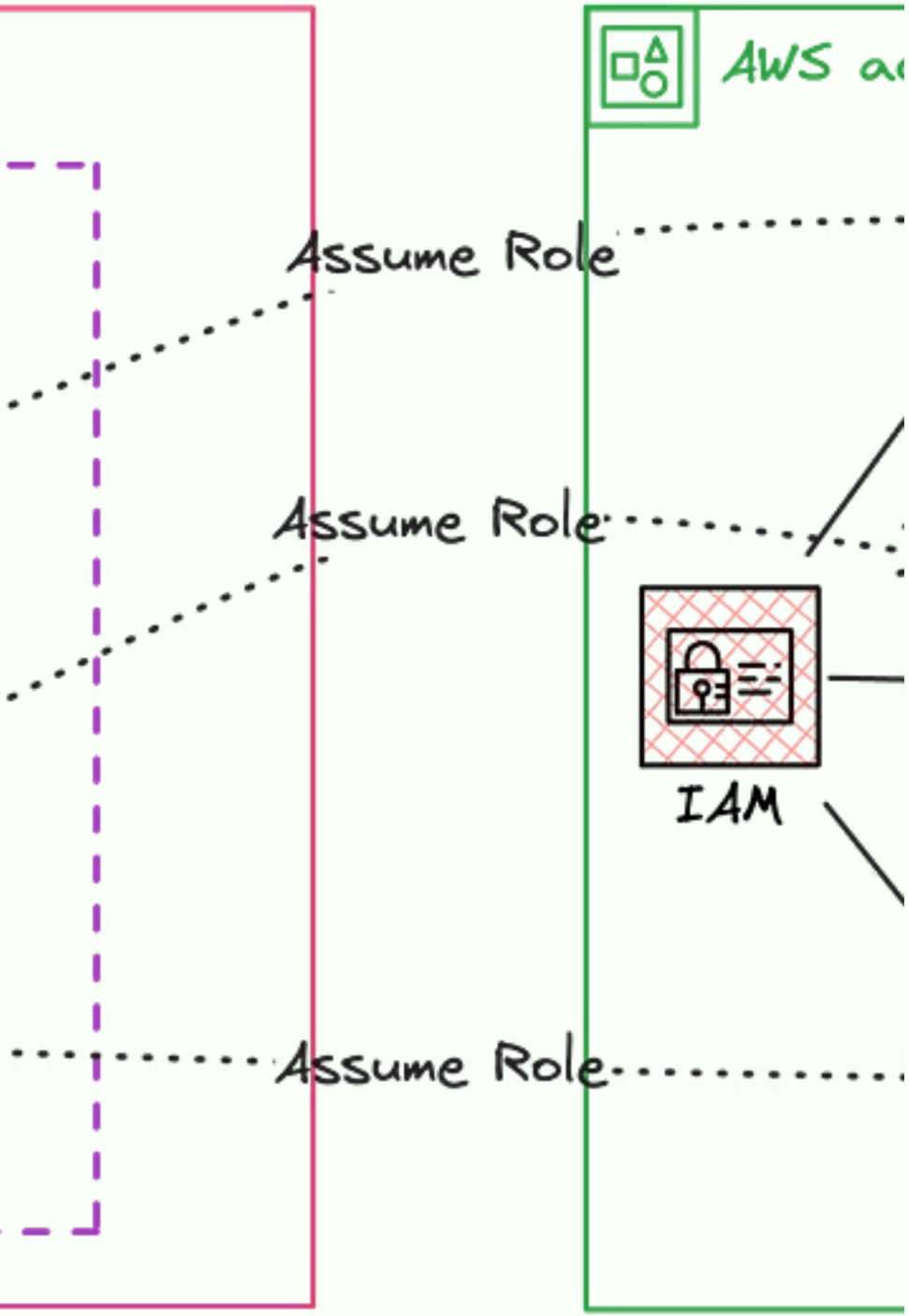
1. Principal Permission

IAM policy allows sts:AssumeRole. Resource is role in trusting account.

2. Role Trust Policy

Trust policy allows principal from trusted account. Principal matches exactly.

If BOTH true → Access allowed. If EITHER false → Access denied.



Role-Based Cross-Account Access

Trust Policies and AssumeRole Flow

```
{  "Version": "2012-10-17",  "Statement": [{    "Effect": "Allow",    "Principal": {      "AWS":
"arn:aws:iam::TRUSTED-ACCOUNT:root"    },    "Action": "sts:AssumeRole",    "Condition": {      "StringEquals": {
  "sts:ExternalId": "unique-external-id-12345"      }    }  }]}
```

1. **User Requests**

   User in Account B calls AssumeRole API

2. **AWS Validates**

   Checks trust policy and external ID

3. **Credentials Issued**

   Temporary credentials returned

4. **Access Granted**

   User accesses resources in Account A

External IDs prevent the confused deputy problem where a third party might trick your role into performing actions on their behalf.

Cross-Account Security Best Practices

Use External IDs

Always use external IDs for third-party access. Prevents confused deputy attacks. Generate unique IDs per partner.

Require MFA

Add MFA condition to trust policies for sensitive operations. Adds extra security layer.

Limit Session Duration

Set maximum session duration to minimum needed. Default 1 hour, maximum 12 hours.

Monitor and Audit

Use CloudTrail to track all AssumeRole calls. Set up alerts for unusual patterns.

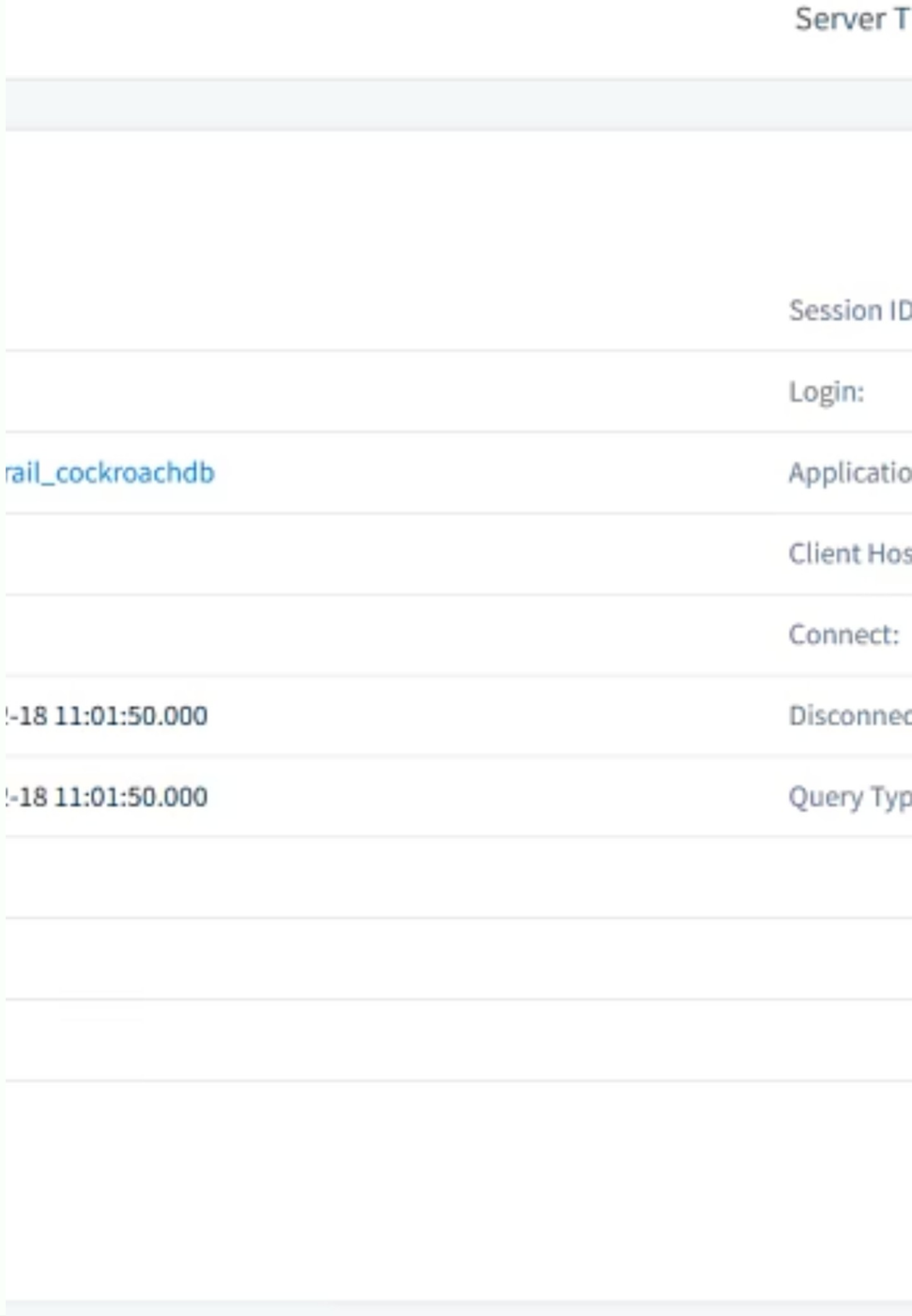
Security Benefits

Compartmentalization: Breach in Account B doesn't access Account A

Blast Radius Reduction: Limited to specific role permissions

Audit Trails: Clear separation of access patterns

Compliance: Can isolate sensitive workloads



Role-Based Access Control (RBAC)

Traditional Permission Model

RBAC assigns permissions based on roles. Each role has specific permissions. Users are assigned to roles based on job function.

1

Developer Role

EC2 full access, RDS read-only, S3 specific buckets

2

QA Role

Read-only access to staging environment resources

3

Operations Role

Full access to production, CloudWatch, monitoring

4

Security Role

IAM management, security services, audit logs

RBAC Challenges

Role Explosion

As organization grows, number of roles multiplies. Each team, project, environment needs separate roles.

Maintenance Burden

Every permission change requires role update. Difficult to scale across large organizations.

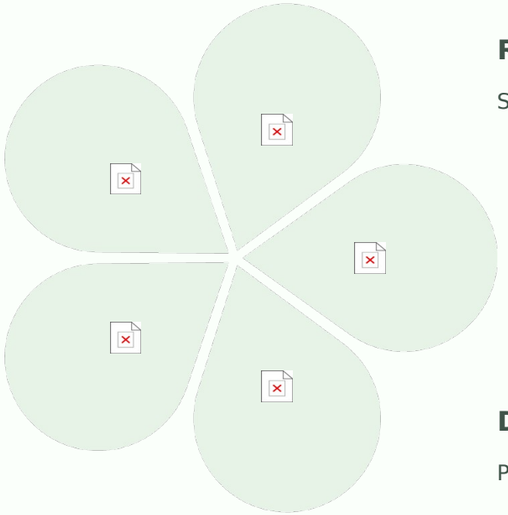
Attribute-Based Access Control (ABAC)

Dynamic Permissions Using Tags

ABAC uses attributes (tags) to determine permissions dynamically. Instead of creating separate roles, permissions are based on matching attributes between principal and resource.

User Attributes

Department, team, location from IdP



Resource Tags

Same attributes on AWS resources

Attribute Matching

Policy allows access when attributes match

Scales Easily

No new roles needed for new teams

Dynamic Access

Permissions adjust automatically

```
{  "Effect": "Allow",  "Action": "ec2:*",  "Resource": "*",  "Condition": {    "StringEquals": {      "ec2:ResourceTag/Department": "${aws:PrincipalTag/Department}"    }  }}
```

When user changes teams, update attribute in directory. No IAM policy changes needed. Access automatically adjusts.

RBAC vs ABAC Comparison

Aspect	RBAC	ABAC
Permission Model	Role-based	Attribute-based
Scalability	Role explosion at scale	Scales with tags
Maintenance	High (many roles)	Low (fewer policies)
Flexibility	Static roles	Dynamic permissions
Complexity	Simple to understand	More complex setup
Best For	Small teams, simple structures	Large orgs, dynamic teams

When to Use RBAC

Small organizations with stable team structures. Clear, well-defined roles. Simple permission requirements.

When to Use ABAC

Large organizations with many teams. Frequent team changes. Need to scale permissions without creating new roles.

Hybrid Approach

Use RBAC for core roles. Add ABAC for fine-grained, dynamic permissions. Best of both worlds.